RL LAB POLICY ITERATION

UNIQ ID: E0124092 DATE: 15/09/2026

```
import numpy as np
import gym

env = gym.make("FrozenLake-v1", is_slippery=False)
print("Number of states: ", env.observation_space.n)
print("Number of actions: ", env.action_space.n)

def policy_evaluation(policy, value_table, env, gamma=0.9, theta=1e-10):
    while True:
        delta = 0
        for state in range(env.observation_space.n):
            v = 0
            action = policy[state]
            # FIX: Added .unwrapped
            for prob, next_state, reward, done in env.unwrapped.P[state]
[action]:
                v += prob * (reward + gamma * value_table[next_state])
            delta = max(delta, abs(value_table[state] - v))
            value_table[state] = v
        if delta < theta:
            break
    return value_table

def policy_improvement(value_table, policy, env, gamma=0.9):
    policy_stable = True
    for state in range(env.observation_space.n):
        old_action = policy[state]
        action_values = np.zeros(env.action_space.n)
        for action in range(env.action_space.n):
            # FIX: Added .unwrapped
            for prob, next_state, reward, done in env.unwrapped.P[state]
[action]:
                action_values[action] += prob * (reward + gamma *
value_table[next_state])
        policy[state] = np.argmax(action_values)
        if old_action != policy[state]:
            policy_stable = False
    return policy, policy_stable

def policy_iteration(env, gamma=0.9, theta=1e-10):
    policy = np.random.choice(env.action_space.n,
size=env.observation_space.n)
    value_table = np.zeros(env.observation_space.n)
    while True:
        value_table = policy_evaluation(policy, value_table, env, gamma,
theta)
        policy, policy_stable = policy_improvement(value_table, policy, env,
gamma)
```

```
        if policy_stable:
            return policy, value_table

optimal_policy, optimal_value = policy_iteration(env, gamma=0.9, theta=1e-10)

print("\nOptimal Policy:")
print(optimal_policy.reshape(4,4)) # Optional: Reshaped for better visibility
print("\nOptimal Value Function:")
print(optimal_value.reshape(4,4))
```

```
Number of states:  16
Number of actions:  4

Optimal Policy:
[[1 2 1 0]
 [1 0 1 0]
 [2 1 1 0]
 [0 2 2 0]]

Optimal Value Function:
[[0.59049 0.6561  0.729   0.6561 ]
 [0.6561  0.      0.81    0.     ]
 [0.729   0.81    0.9     0.     ]
 [0.      0.9     1.      0.     ]]
```